

Queues Data Structure in Python



Follow @techie_programmer for more Programming content!

Definition

A **Queue** is a linear data structure that follows the **FIFO (First-In-First-Out)** principle. The first element added to the queue will be the first one to be removed.

Think of it like a line of people waiting at a ticket counter - the person who arrives first gets served first.

How It Works

A queue has two ends:

- **Rear (Back):** Where elements are added (enqueue operation)
- **Front:** Where elements are removed (dequeue operation)

Visual Representation:

```
Front → [10] [20] [30] [40] ← Rear
         ↑           ↑
        Remove here  Add here
```

Core Operations

1. Enqueue (Add)

Adds an element to the rear of the queue.

Time Complexity: $O(1)$

2. Dequeue (Remove)

Removes and returns the element from the front of the queue.

Time Complexity: $O(1)$ or $O(n)$ depending on implementation

3. Peek/Front

Returns the front element without removing it.

Time Complexity: $O(1)$

4. IsEmpty

Checks if the queue is empty.

Time Complexity: $O(1)$

5. Size

Returns the number of elements in the queue.

Time Complexity: $O(1)$

Implementation Methods

Python Implementation

```
class Queue:
    def __init__(self):
        self.items = []
```

```

def enqueue(self, element):
    """Add an element to the rear of the queue"""
    self.items.append(element)

def dequeue(self):
    """Remove and return the front element"""
    if self.is_empty():
        raise IndexError("Queue is empty")
    return self.items.pop(0)

def peek(self):
    """Return the front element without removing it"""
    if self.is_empty():
        raise IndexError("Queue is empty")
    return self.items[0]

def is_empty(self):
    """Check if the queue is empty"""
    return len(self.items) == 0

def size(self):
    """Return the number of elements in the queue"""
    return len(self.items)

# Example usage
queue = Queue()
queue.enqueue(10)
queue.enqueue(20)
queue.enqueue(30)
print(queue.dequeue()) # Output: 10
print(queue.peek())    # Output: 20

```

Java Implementation

```

class Queue {
    private int[] items;
    private int front;
    private int rear;
    private int size;
    private int capacity;

    public Queue(int capacity) {
        this.capacity = capacity;
        this.items = new int[capacity];
        this.front = 0;
        this.rear = -1;
        this.size = 0;
    }

    public void enqueue(int element) {
        if (isFull()) {
            throw new RuntimeException("Queue is full");
        }
        rear = (rear + 1) % capacity;
        items[rear] = element;
        size++;
    }

    public int dequeue() {
        if (isEmpty()) {
            throw new RuntimeException("Queue is empty");
        }
        int element = items[front];
        front = (front + 1) % capacity;
        size--;
        return element;
    }

    public int peek() {

```

```

        if (isEmpty()) {
            throw new RuntimeException("Queue is empty");
        }
        return items[front];
    }

    public boolean isEmpty() {
        return size == 0;
    }

    public boolean isFull() {
        return size == capacity;
    }

    public int size() {
        return size;
    }

    public static void main(String[] args) {
        Queue queue = new Queue(5);
        queue.enqueue(10);
        queue.enqueue(20);
        queue.enqueue(30);
        System.out.println(queue.dequeue()); // Output: 10
        System.out.println(queue.peek());    // Output: 20
    }
}

```

C Implementation

```

#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

```

```

#define MAX_SIZE 100

typedef struct {
    int items[MAX_SIZE];
    int front;
    int rear;
    int size;
} Queue;

void initialize(Queue* queue) {
    queue->front = 0;
    queue->rear = -1;
    queue->size = 0;
}

bool isEmpty(Queue* queue) {
    return queue->size == 0;
}

bool isFull(Queue* queue) {
    return queue->size == MAX_SIZE;
}

void enqueue(Queue* queue, int element) {
    if (isFull(queue)) {
        printf("Queue is full\n");
        return;
    }
    queue->rear = (queue->rear + 1) % MAX_SIZE;
    queue->items[queue->rear] = element;
    queue->size++;
}

int dequeue(Queue* queue) {
    if (isEmpty(queue)) {
        printf("Queue is empty\n");
        return -1;
    }

```

```

    }
    int element = queue->items[queue->front];
    queue->front = (queue->front + 1) % MAX_SIZE;
    queue->size--;
    return element;
}

int peek(Queue* queue) {
    if (isEmpty(queue)) {
        printf("Queue is empty\n");
        return -1;
    }
    return queue->items[queue->front];
}

int size(Queue* queue) {
    return queue->size;
}

int main() {
    Queue queue;
    initialize(&queue);

    enqueue(&queue, 10);
    enqueue(&queue, 20);
    enqueue(&queue, 30);

    printf("%d\n", dequeue(&queue)); // Output: 10
    printf("%d\n", peek(&queue));    // Output: 20

    return 0;
}

```

Alternative Python Implementation Using collections.deque (Efficient)

```
from collections import deque

class Queue:
    def __init__(self):
        self.queue = deque()

    def enqueue(self, item):
        """Add item to rear of queue"""
        self.queue.append(item)

    def dequeue(self):
        """Remove and return front item"""
        if self.is_empty():
            return "Queue is empty"
        return self.queue.popleft() # O(1) - efficient

    def peek(self):
        """Return front item without removing"""
        if self.is_empty():
            return "Queue is empty"
        return self.queue[0]

    def is_empty(self):
        """Check if queue is empty"""
        return len(self.queue) == 0

    def size(self):
        """Return size of queue"""
        return len(self.queue)

# Example usage
```

```

q = Queue()
q.enqueue(10)
q.enqueue(20)
q.enqueue(30)
print("Dequeue:", q.dequeue()) # 10
print("Peek:", q.peek())      # 20
print("Size:", q.size())      # 2

```

Time Complexity Summary

Operation	List Implementation	deque Implementation
Enqueue	O(1)	O(1)
Dequeue	O(n)	O(1)
Peek	O(1)	O(1)
IsEmpty	O(1)	O(1)
Size	O(1)	O(1)

Space Complexity

O(n) where n is the number of elements in the queue.

Key Points to Remember

1. Queue follows FIFO principle
2. Use `collections.deque` for efficient queue operations in Python
3. `list.pop(0)` is O(n) - avoid for large queues
4. `deque.popleft()` is O(1) - recommended
5. Always check if queue is empty before dequeue/peek operations

6. Java and C implementations use circular arrays for efficiency